ĐẠI HỌC HOA SEN

TÔN TRỌNG SỰ KHÁC BIỆT

Pandas: Reading & Writing Data

Faculty of Information Technology

ĐẠI HỌC HOA SEN

Objectives

- ✓ Reading Data in CSV or Text Files
- ✓ Reading and Writing HTML Files
- ✓ Reading Data from XML
- ✓ Reading and Writing Data on Microsoft Excel Files
- ✓ JSON Data
- ✓ Pickle—Python Object Serialization

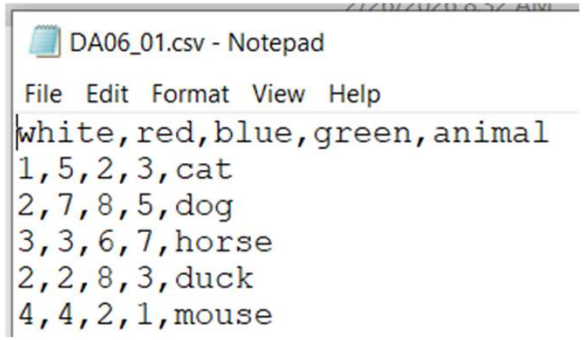
Reading Data in CSV or Text Files

- The most common operation of a person approaching data analysis is to read the data contained in a CSV file, or at least in a text file.
- But before you start dealing with files, you need to import the following libraries.

```
import numpy as np
import pandas as pd
import io
```

```
from google.colab import files
uploaded = files.upload()
```

- DA06_01.csv: This file is comma-delimited, you can use the **read_csv()** function to read its content and convert it to a dataframe object.




```
csvframe = pd.read_csv('DA06_01.csv')
csvframe
```

	white	red	blue	green	animal
0	1	5	2	3	cat
1	2	7	8	5	dog
2	3	3	6	7	horse
3	2	2	8	3	duck
4	4	4	2	1	mouse

- DA06_02.csv: the tabulated data begin directly in the first line (no headers)
  - read_table() – read_csv()

DA06_02.csv - Notepad

File Edit Format View Help

```
1,5,2,3,cat
2,7,8,5,dog
3,3,6,7,horse
2,2,8,3,duck
4,4,2,1,mouse
```

```
#Upload DA06_02.csv file
from google.colab import files
uploaded = files.upload()
```

Choose Files DA06_02.csv

DA06_02.csv(text/csv) - 65 bytes, last modified: 2/26/2026 - 100% done

Saving DA06_02.csv to DA06_02.csv

```
pd.read_table('DA06_02.csv')
```

	1,5,2,3,cat
0	2,7,8,5,dog
1	3,3,6,7,horse
2	2,2,8,3,duck
3	4,4,2,1,mouse

```
pd.read_table('DA06_02.csv',sep=',')
```

	1	5	2	3	cat
0	2	7	8	5	dog
1	3	3	6	7	horse
2	2	2	8	3	duck
3	4	4	2	1	mouse

```
pd.read_csv('DA06_02.csv')
```

	1	5	2	3	cat
0	2	7	8	5	dog
1	3	3	6	7	horse
2	2	2	8	3	duck
3	4	4	2	1	mouse

```
pd.read_csv('DA06_02.csv', header=None)
```

	0	1	2	3	4
0	1	5	2	3	cat
1	2	7	8	5	dog
2	3	3	6	7	horse
3	2	2	8	3	duck
4	4	4	2	1	mouse

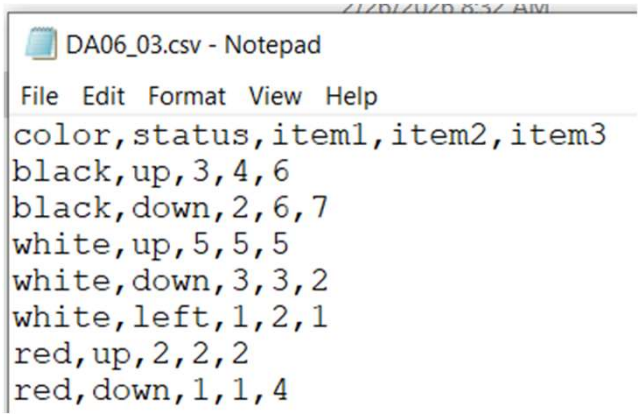
```
pd.read_csv('DA06_02.csv')
```

	1	5	2	3	cat
0	2	7	8	5	dog
1	3	3	6	7	horse
2	2	2	8	3	duck
3	4	4	2	1	mouse

```
# specify the names directly by assigning a list of labels to the names option
pd.read_csv('DA06_02.csv', names=['white','red','blue','green','animal'])
```

	white	red	blue	green	animal
0	1	5	2	3	cat
1	2	7	8	5	dog
2	3	3	6	7	horse
3	2	2	8	3	duck
4	4	4	2	1	mouse

- In more complex cases: create a dataframe with a hierarchical structure by reading a CSV file,
- Extend the functionality of the read_csv() function by adding the **index_col option**, assigning all the columns to be converted into indexes.




```
#Upload DA06_03.csv file
from google.colab import files
uploaded = files.upload()
```

Choose Files DA06_03.csv
DA06_03.csv(text/csv) - 140 bytes, last modified: 2/26/2026 - 100% done
Saving DA06_03.csv to DA06_03.csv

```
pd.read_csv('DA06_03.csv', index_col=['color','status'])
```

		item1	item2	item3
color	status			
black	up	3	4	6
	down	2	6	7
white	up	5	5	5
	down	3	3	2
	left	1	2	1
red	up	2	2	2
	down	1	1	4

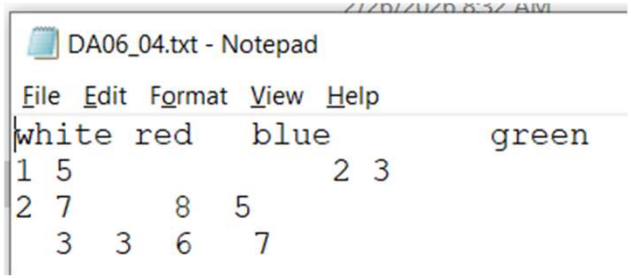
Using Regexp to Parse TXT Files

- Regexp (regular expression): separating values in a TXT file, consider situations where values are separated by **spaces or tabs in an unpredictable order**. In such cases, simple delimiters (like a single space or comma) are not sufficient.
- Regular expressions allow you to define flexible separation rules. For example:
  - \s represents **any whitespace character** (space, tab, etc.).
  - \t specifically represents a **tab character**.
  - * is a quantifier meaning **zero or more occurrences** of the preceding character.
- By using a pattern like \s*, you indicate that there may be **multiple whitespace characters** (spaces or tabs) between values. This makes regexp especially useful when handling inconsistent or messy text data.

- Metacharacters

.	Single character, except newline
\d	Digit
\D	Non-digit character
\s	Whitespace character
\S	Non-whitespace character
\n	New line character
\t	Tab character
\uxxxx	Unicode character specified by the hexadecimal number xxxx

- Take for example an extreme case in which you have the values separated by **tabs** or **spaces** in random order




```
#Upload DA06_04.txt file
from google.colab import files
uploaded = files.upload()
```

Choose Files DA06_04.txt

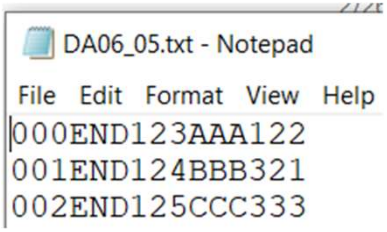
DA06_04.txt(text/plain) - 54 bytes, last modified: 2/26/2026 - 100% done

Saving DA06_04.txt to DA06_04.txt

```
pd.read_table('DA06_04.txt', sep=r'\s+', engine='python')
```

	white	red	blue	green
0	1	5	2	3
1	2	7	8	5
2	3	3	6	7

- To extract the numeric part from a TXT file, in which there is a sequence of characters with **numerical values** and the **literal characters** are completely fused.



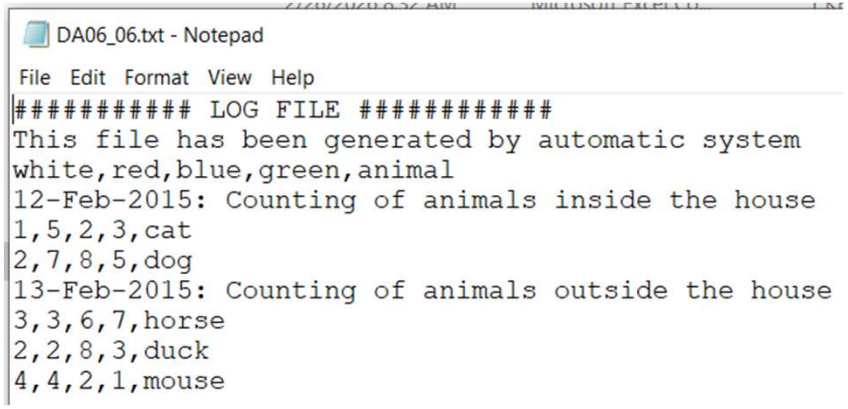

```
#Upload DA06_05.txt file
uploaded = files.upload()
```

```
Choose Files DA06_05.txt
DA06_05.txt(text/plain) - 47 bytes, last modified: 2/26/2026 - 100% done
Saving DA06_05.txt to DA06_05.txt
```

```
pd.read_table('DA06_05.txt', sep=r'\D+', header=None, engine='python')
```

	0	1	2
0	0	123	122
1	1	124	321
2	2	125	333

- When parsing files, you may need to exclude certain lines, such as headers or comments. This can be done using the skiprows option.
  - To skip the first N lines, use:
    skiprows = 5 → skips the first five lines.
  - To skip specific line numbers, use a list:
    skiprows = [5] → skips only the fifth line.
- Be careful with the syntax, as a number skips multiple initial lines, while a list skips specific lines.




```
#Upload DA06_06.txt file
uploaded = files.upload()
```

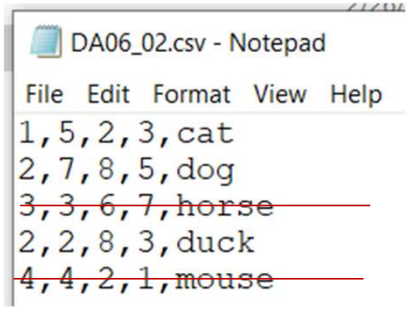
Choose Files No file chosen Upload widget is only availabl
Saving DA06_06.txt to DA06_06.txt

```
pd.read_table('DA06_06.txt',sep=r',',skiprows=[0,1,3,6])
```

	white	red	blue	green	animal
0	1	5	2	3	cat
1	2	7	8	5	dog
2	3	3	6	7	horse
3	2	2	8	3	duck
4	4	4	2	1	mouse

Reading TXT Files Into Parts

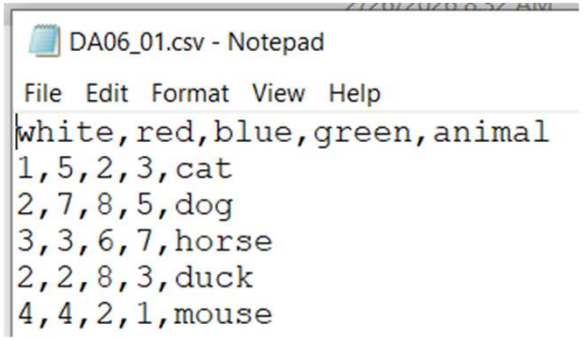
- When working with large files or when only part of the data is needed, you can read the file in chunks instead of loading everything at once.
- Using the skiprows and nrows options allows you to control which portion of the file is read:
  - skiprows = n → defines the starting line (skips the first n lines).
  - nrows = i → specifies how many lines to read after that point.
- This approach improves efficiency and allows selective parsing of large datasets.




```
pd.read_csv('DA06_02.csv',skiprows=[2],nrows=3,header=None)
```

	0	1	2	3	4
0	1	5	2	3	cat
1	2	7	8	5	dog
2	2	2	8	3	duck

- Another common task when processing text data is **splitting it into portions** and performing specific operations on each portion iteratively.
- For example, you might group data every three rows, calculate the sum of a column within each group, and store those sums in a new series.
- Although this example is simple, it demonstrates the basic mechanism of processing data **portion by portion**, which can later be applied to more complex and practical scenarios.



	white	red	blue	green	animal
0	1	5	2	3	cat
1	2	7	8	5	dog
2	3	3	6	7	horse
3	2	2	8	3	duck
4	4	4	2	1	mouse

```
out = pd.Series(dtype='float64')
```

```
i = 0
```

```
pieces = pd.read_csv('DA06_01.csv', chunksize=3)
```

```
for piece in pieces:
  out.at[i] = piece['white'].sum()
  i = i + 1
```

```
out
```

	0
0	6
1	6

dtype: int64

Writing Data in CSV

- Besides reading data from files, it is also common to **write processed data to a new file**. For example, you can export the contents of a dataframe to a CSV file.
- In Python (pandas), this is done using the to_csv() function, which takes the **output file name** as an argument and saves the dataframe to that file.

```
frame = pd.DataFrame(np.arange(16).reshape((4,4)),
  index=['red', 'blue', 'yellow', 'white'],
  columns=['ball','pen','pencil','paper'])
```

```
frame
```

	ball	pen	pencil	paper
red	0	1	2	3
blue	4	5	6	7
yellow	8	9	10	11
white	12	13	14	15

Next steps: Generate code with frame | New interactive sheet

```
frame.to_csv('DA06_07.csv')
```

```
files.download('DA06_07.csv')
```

DA06_07.csv - Notepad

File Edit Format View Help

```
,ball,pen,pencil,paper
red,0,1,2,3
blue,4,5,6,7
yellow,8,9,10,11
white,12,13,14,15
```

- One point to remember when writing files is that NaN values present in a data structure are shown as empty fields in the file

```
frame3 = pd.DataFrame([[6,np.nan,np.nan,6,np.nan],
 [np.nan,np.nan,np.nan,np.nan,np.nan],
 [np.nan,np.nan,np.nan,np.nan,np.nan],
 [20,np.nan,np.nan,20.0,np.nan],
 [19,np.nan,np.nan,19.0,np.nan]],
  index=['blue','green','red','white','yellow'],
  columns=['ball','mug','paper','pen','pencil'])
```

```
frame3.to_csv('DA06_08.csv')
```

```
frame3
```

	ball	mug	paper	pen	pencil
blue	6.0	NaN	NaN	6.0	NaN
green	NaN	NaN	NaN	NaN	NaN
red	NaN	NaN	NaN	NaN	NaN
white	20.0	NaN	NaN	20.0	NaN
yellow	19.0	NaN	NaN	19.0	NaN

```
files.download('DA06_08.csv')
```

DA06_08.csv - Notepad

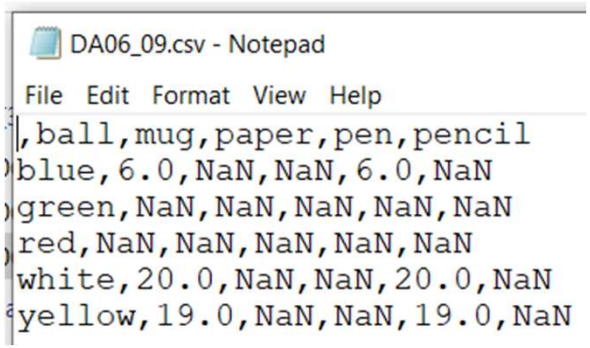
File Edit Format View Help

```
,ball,mug,paper,pen,pencil
blue,6.0,,,6.0,
green,,,,,
red,,,,,
white,20.0,,,20.0,
yellow,19.0,,,19.0,
```

- can replace this empty field with a value to your liking using the **na_rep** option in the to_csv() function. Common values include NULL, 0, or the same NaN

```
frame3.to_csv('DA06_09.csv', na_rep ='NaN')
```

```
files.download('DA06_09.csv')
```



Reading and Writing HTML Files

- pandas provides the corresponding pair of I/O API functions for the HTML format.
  - read_html()
  - to_html()

Writing Data in HTML

- A dataframe can be easily converted into an HTML table using the to_html() function. This function automatically transforms the dataframe's internal structure into proper HTML tags such as <TH>, <TR>, and <TD>, preserving any hierarchical structure.
- No knowledge of HTML is required to use this feature. It is especially useful when creating web pages, as even large and complex dataframes can be quickly converted into web-ready tables.

```
frame = pd.DataFrame(np.arange(4).reshape(2,2))
print(frame.to_html())
```

```
<table border="1" class="dataframe">
  <thead>
    <tr style="text-align: right;">
      <th></th>
      <th>0</th>
      <th>1</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th>0</th>
      <td>0</td>
      <td>1</td>
    </tr>
    <tr>
      <th>1</th>
      <td>2</td>
      <td>3</td>
    </tr>
  </tbody>
</table>
```

- The next example demonstrates how a dataframe can be automatically rendered as a table inside an HTML file. In this case, the dataframe is more complex than the previous example, including index labels and column names, which are also correctly displayed in the generated HTML table.

```
frame = pd.DataFrame( np.random.random((4,4)),
                      index = ['white','black','red','blue'],
                      columns = ['up','down','right','left'])
frame
```

	up	down	right	left
white	0.977597	0.458318	0.005086	0.720185
black	0.298750	0.811390	0.315979	0.501646
red	0.777665	0.106061	0.633056	0.899390
blue	0.419109	0.174289	0.694105	0.117476

```
s = ['<HTML>']
s.append('<HEAD><TITLE>My DataFrame</TITLE></HEAD>')
s.append('<BODY>')
s.append(frame.to_html())
s.append('</BODY></HTML>')
html = ''.join(s)
```

```
html_file = open('myFrame.html','w')
html_file.write(html)
html_file.close()
```

```
files.download('myFrame.html')
```

Reading Data from an HTML File

- Just as pandas can generate HTML tables from a dataframe, it can also perform the reverse operation. **The read_html()** function parses an HTML page, searches for <table> elements, and converts them into pandas dataframes for analysis.
- Importantly, read_html() always returns a **list of dataframes**, even if only one table is found. The HTML source can come from different locations, such as a local HTML file or other supported sources.

```
web_frames = pd.read_html('myFrame.html')
web_frames[0]
```

	Unnamed: 0	up	down	right	left
0	white	0.977597	0.458318	0.005086	0.720185
1	black	0.298750	0.811390	0.315979	0.501646
2	red	0.777665	0.106061	0.633056	0.899390
3	blue	0.419109	0.174289	0.694105	0.117476

Reading Data from XML

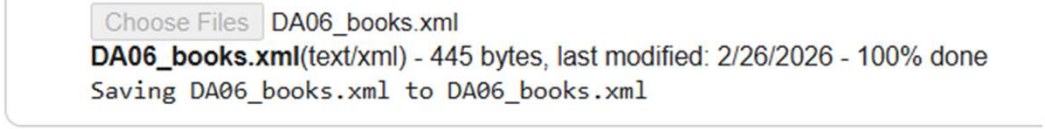
- Although pandas does not include a specific I/O function for XML format in its main API list, XML remains an important format for structured data.
- To work with XML files, Python provides other libraries, such as **lxml**, which is known for its high performance when parsing large XML files. By using libraries like lxml alongside pandas, you can parse XML data and then convert the extracted information into a dataframe for analysis.

DA06_books.xml - Notepad

File Edit Format View Help

```
<?xml version="1.0"?>
<Catalog>
        <Book id="ISBN9872122367564">
                <Author>Ross, Mark</Author>
                <Title>XML Cookbook</Title>
                <Genre>Computer</Genre>
                <Price>23.56</Price>
                <PublishDate>2014-22-01</PublishDate>
        </Book>
        <Book id="ISBN9872122367564">
                <Author>Bracket, Barbara</Author>
                <Title>XML for Dummies</Title>
                <Genre>Computer</Genre>
                <Price>35.95</Price>
                <PublishDate>2014-12-16</PublishDate>
        </Book>
</Catalog>
```

```
uploaded = files.upload()
```




```
from lxml import objectify
xml = objectify.parse('DA06_books.xml')
xml
```

```
<lxml.etree._ElementTree at 0x79f58c8d4380>
```

- To navigate in this tree structure, so as to select element by element, you must first define the root. You can do this with the getroot() function.

```
root = xml.getroot()
```

- To select them, simply write the various separate tags with points, reflecting in a certain way the hierarchy of nodes in the tree.

```
root.Book.Author
```

```
'Ross, Mark'
```

```
root.Book.PublishDate
```

```
'2014-22-01'
```

- Access various elements at the same time using getchildren(): get all the child nodes of the reference element.

```
root.getchildren()
```

```
[<Element Book at 0x79f58b541440>, <Element Book at 0x79f58986a200>]
```

```
[child.tag for child in root.Book.getchildren()]
```

```
['Author', 'Title', 'Genre', 'Price', 'PublishDate']
```

```
[child.text for child in root.Book.getchildren()]
```

```
['Ross, Mark', 'XML Cookbook', 'Computer', '23.56', '2014-22-01']
```

- Although you can navigate the lxml.etree tree structure, the main goal is to **convert the XML data into a dataframe**. To achieve this, you can define a function that analyzes the contents of the etree and extracts the relevant elements, filling the dataframe **row by row** with the parsed data.

```
def etree2df(root):
    column_names = []
    for i in range(0, len(root.getchildren()[0].getchildren())):
        column_names.append(root.getchildren()[0].getchildren()[i].tag)

    rows_list = [] # Collect rows in a list
    for j in range(0, len(root.getchildren())):
        obj = root.getchildren()[j].getchildren()
        texts = []
        for k in range(0, len(column_names)):
            texts.append(obj[k].text)
        row = dict(zip(column_names, texts))
        rows_list.append(row) # Append dictionary to list

    # Create DataFrame from list of dictionaries
    xmlframe = pd.DataFrame(rows_list)
    return xmlframe
```

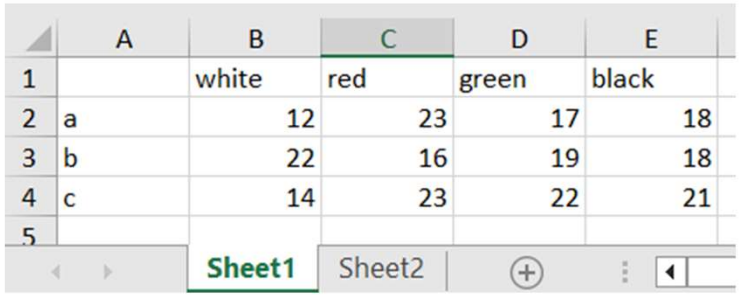
```
etree2df(root)
```

	Author	Title	Genre	Price	PublishDate
0	Ross, Mark	XML Cookbook	Computer	23.56	2014-22-01
1	Bracket, Barbara	XML for Dummies	Computer	35.95	2014-12-16

Reading and Writing Data on Microsoft Excel Files

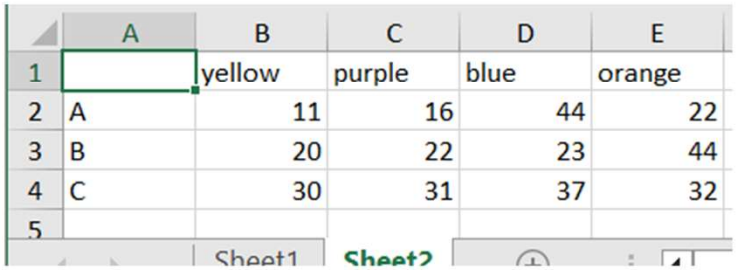
- While CSV files are commonly used for storing tabular data, data is often also stored in **Excel spreadsheets**.
- To handle Excel files, pandas provides dedicated functions:
  - read_excel() → to read data from an Excel file into a dataframe.
  - to_excel() → to write a dataframe into an Excel file.
- These functions allow easy import and export of Excel-based data for analysis.

- To read the data contained in the XLS file and convert it into a dataframe, you only have to use the read_excel() function.



	A	B	C	D	E
1		white	red	green	black
2	a	12	23	17	18
3	b	22	16	19	18
4	c	14	23	22	21
5					

Sheet1 Sheet2



	A	B	C	D	E
1		yellow	purple	blue	orange
2	A	11	16	44	22
3	B	20	22	23	44
4	C	30	31	37	32
5					

Sheet1 Sheet2

```
pd.read_excel('DA06_data.xlsx')
```

	Unnamed: 0	white	red	green	black
0	a	12	23	17	18
1	b	22	16	19	18
2	c	14	23	22	21

```
pd.read_excel('DA06_data.xlsx','Sheet2')
```

	Unnamed: 0	yellow	purple	blue	orange
0	A	11	16	44	22
1	B	20	22	23	44
2	C	30	31	37	32

```
uploaded = files.upload()
```

Choose Files DA06_data.xlsx

DA06_data.xlsx(application/vnd.openxmlformats-officedocument.spreadsheetml.sheet) - 16447 bytes, last modified: 2/26/2026 - 100% done

```
Saving DA06_data.xlsx to DA06_data.xlsx
```

sự khác biệt

- To convert a dataframe into an Excel spreadsheet

```
frame = pd.DataFrame(np.random.random((4,4)),
 index = ['exp1','exp2','exp3','exp4'],
 columns = ['Jan2015','Fab2015','Mar2015','Apr2005'])
```

```
frame
```

	Jan2015	Fab2015	Mar2015	Apr2005
exp1	0.768078	0.730063	0.962594	0.826636
exp2	0.957576	0.576487	0.761288	0.205795
exp3	0.879635	0.119390	0.102865	0.109711
exp4	0.022037	0.077078	0.667422	0.624689

```
frame.to_excel('DA06_data2.xlsx')
```

```
files.download('DA06_data2.xlsx')
```

	A	B	C	D	E
1		Jan2015	Fab2015	Mar2015	Apr2005
2	exp1	0.768078	0.730063	0.962594	0.826636
3	exp2	0.957576	0.576487	0.761288	0.205795
4	exp3	0.879635	0.11939	0.102865	0.109711
5	exp4	0.022037	0.077078	0.667422	0.624689

JSON Data

- **JSON (JavaScript Object Notation)** is one of the most widely used data formats, especially for web data transmission. Its structure is highly flexible, but it differs from the traditional tabular format commonly used in data analysis.
- Pandas provides two main functions to handle JSON within its I/O API:
  - read_json() → to import JSON data into a dataframe
  - to_json() → to export dataframe data into JSON format
- The section first explains these basic functions, then presents a more realistic example involving structured JSON data similar to real-world applications.

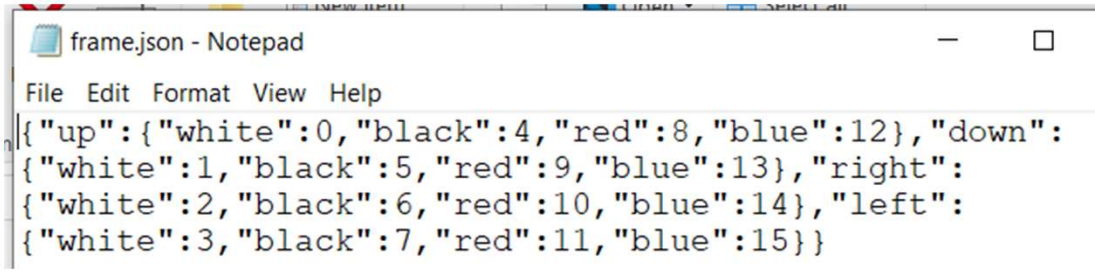
- A common use case is converting a **dataframe into a JSON file**. To do this, first define the dataframe, then use the to_json() function and pass the desired output file name as an argument.
- This exports the dataframe's data into JSON format, making it suitable for web applications or data exchange.

```
frame = pd.DataFrame(np.arange(16).reshape(4,4),
                     index=['white','black','red','blue'],
                     columns=['up','down','right','left'])
frame.to_json('frame.json')
```

```
pd.read_json('frame.json')
```

	up	down	right	left
white	0	1	2	3
black	4	5	6	7
red	8	9	10	11
blue	12	13	14	15

```
files.download('frame.json')
```



- In most real-world cases, JSON files do not follow a simple tabular structure. Instead, they often contain nested dictionaries or lists. To use this data in a dataframe, you must convert it into a tabular format — a process known as normalization.
- Pandas provides the **json_normalize() function**, which transforms nested dictionaries or lists into a flat table structure suitable for data analysis.

DA06_books.json - Notepad

File Edit Format View Help

```
[{"writer": "Mark Ross",
"nationality": "USA",
"books": [
        {"title": "XML Cookbook", "price": 23.56},
        {"title": "Python Fundamentals", "price": 50.70},
        {"title": "The NumPy library", "price": 12.30}
]
},
{"writer": "Barbara Bracket",
"nationality": "UK",
"books": [
        {"title": "Java Enterprise", "price": 28.60},
        {"title": "HTML5", "price": 31.35},
        {"title": "Python for Dummies", "price": 28.00}
]
}]
```

```
files.upload()
```

```
from pandas import json_normalize
```

```
import json # Import the standard json module
file = open('DA06_books.json','r')
text = file.read()
text = json.loads(text) # Use Python's built-in json module
```

- apply the json_normalize() function
  - The function extracts all elements associated with the key **"books"** and converts their properties into columns in a dataframe. Nested properties become nested column names, and their corresponding values populate the rows. The dataframe indexes are automatically assigned as an increasing numerical sequence.

```
json_normalize(text,'books')
```

	title	price
0	XML Cookbook	23.56
1	Python Fundamentals	50.70
2	The NumPy library	12.30
3	Java Enterprise	28.60
4	HTML5	31.35
5	Python for Dummies	28.00

- By default, the function may return a dataframe containing only part of the nested information. To include additional data from other keys at the same level, you can specify a list of those keys as an additional argument in the function.
- This allows the resulting dataframe to include extra columns with related information, creating a more complete tabular structure.

```
pd.json_normalize(text,'books',['writer','nationality'])
```

	title	price	writer	nationality
0	XML Cookbook	23.56	Mark Ross	USA
1	Python Fundamentals	50.70	Mark Ross	USA
2	The NumPy library	12.30	Mark Ross	USA
3	Java Enterprise	28.60	Barbara Bracket	UK
4	HTML5	31.35	Barbara Bracket	UK
5	Python for Dummies	28.00	Barbara Bracket	UK

Pickle—Python Object Serialization

- The **pickle** module in Python provides a mechanism for **serialization and deserialization** of Python objects.
  - **Pickling** converts an object's structure into a stream of bytes.
  - **Unpickling** reconstructs the original object from that byte stream, preserving its structure and features.
- In addition to the standard pickle module, Python includes a faster implementation called **_pickle**, written in C, which can be significantly more efficient. Despite performance differences, both modules share nearly identical interfaces.
- Before exploring pandas I/O functions related to this format, it is helpful to understand how the _pickle module works and how to use it directly.

Serialize a Python Object with cPickle

- The data format used by the pickle (or cPickle) module is specific to Python. By default, an ASCII representation is used to represent it, in order to be readable from the human point of view.

```
#import cPickle
import _pickle as pickle
```

```
#create an object sufficiently complex to have an internal data structure,
#for example a dict object
data = { 'color': ['white','red'], 'value': [5, 7]}
```

```
#Now perform a serialization of the data object through
#the dumps() function of the cPickle module.
pickled_data = pickle.dumps(data)
```

```
#see how it serialized the dict object
print(pickled_data)
```

```
b'\x80\x04\x95/\x00\x00\x00\x00\x00\x00\x00}\x94(\x8c\x05color\x94]\x94(\x8c\x05white\x94\x8c\x03red\x94e\x8c\x05value
```

- Once data has been serialized (pickled), it can be easily stored in a file or transmitted through a socket, pipe, or other communication channels.
- After transmission, the original object can be reconstructed through **deserialization** using the loads() function from the _pickle module.

```
nframe = pickle.loads(pickled_data)
```

```
nframe
```

```
{'color': ['white', 'red'], 'value': [5, 7]}
```

Pickling with pandas

- With pandas, pickling and unpickling are simplified. There is no need to explicitly import the pickle or _pickle module, as pandas handles the serialization process internally.
- Additionally, the serialization format used by pandas is not entirely ASCII-based, making it more efficient for storing complex data structures.

```
frame = pd.DataFrame(np.arange(16).reshape(4,4), index = ['up','down','left','right'])
```

```
frame.to_pickle('frame.pkl')
```

```
#To open a PKL file and read the contents
pd.read_pickle('frame.pkl')
```

	0	1	2	3
up	0	1	2	3
down	4	5	6	7
left	8	9	10	11
right	12	13	14	15

Group Lab

- Loading and Writing Data with SQLite
- Loading and Writing Data with PostgreSQL
- Reading and Writing Data with a NoSQL Database: MongoDB


Q&A